

Can LLMs Hack Enterprise Networks?

Autonomous Assumed Breach Penetration-Testing Active Directory Networks

ANDREAS HAPPE, TU Wien, Austria

JÜRGEN CITO, TU Wien, Austria

Traditional enterprise penetration-testing, while critical for validating defenses and uncovering vulnerabilities, is often limited by high operational costs and the scarcity of human expertise. This paper investigates the feasibility and effectiveness of using Large Language Model (LLM)-driven autonomous systems to address these challenges in real-world Active Directory (AD) enterprise networks.

We introduce a novel prototype, *cochise*, designed to employ LLMs to autonomously perform Assumed Breach penetration-testing against enterprise networks. Our system represents the first demonstration of a fully autonomous, LLM-driven framework capable of compromising accounts within a real-life Microsoft Active Directory testbed, the *Game of Active Directory* (GOAD). The evaluation deliberately utilizes GOAD to capture the intricate interactions and sometimes nondeterministic outcomes of live network penetration-testing, moving beyond the limitations of synthetic benchmarks.

We perform our empirical evaluation using five LLMs, comparing reasoning to non-reasoning models as well as including open-weight models. Through comprehensive quantitative and qualitative analysis, incorporating insights from cybersecurity experts, we demonstrate that autonomous LLMs can effectively conduct Assumed Breach simulations. Key findings highlight their ability to dynamically adapt attack strategies, perform inter-context attacks (e.g., web application audits, social engineering, and unstructured data analysis for credentials), and generate scenario-specific attack parameters like realistic password candidates. The prototype also exhibits robust self-correction mechanisms, automatically installing missing tools and rectifying invalid command generations.

Critically, we find that the associated costs are competitive with, and often significantly lower than, those incurred by professional human penetration testers, suggesting a path toward democratizing access to essential security testing for organizations with budgetary constraints. However, our research also illuminates existing limitations, including instances of LLM “going down rabbit holes”, challenges in comprehensive information transfer between planning and execution modules, and critical safety concerns that necessitate human oversight. Our findings lay foundational groundwork for future software engineering research into LLM-driven cybersecurity automation, emphasizing that the prototype’s underlying LLM-driven architecture and techniques are domain-agnostic and hold promise for improving autonomous LLM usage in broader software engineering domains. The source code, traces, and analyzed logs are open-sourced to foster collective cybersecurity and future research.

CCS Concepts: • **Computing methodologies** → *Planning under uncertainty*; • **Security and privacy** → **Software and application security**; **Systems security**.

Additional Key Words and Phrases: Security Capability Evaluation, Large Language Models, Enterprise Networks

ACM Reference Format:

Andreas Happe and Jürgen Cito. 2025. Can LLMs Hack Enterprise Networks?: Autonomous Assumed Breach Penetration-Testing Active Directory Networks. *ACM Trans. Softw. Eng. Methodol.* 1, 1, Article 1 (January 2025), 56 pages. <https://doi.org/10.1145/3766895>

1 Introduction

Recent advancements in artificial intelligence have sparked significant interest in leveraging off-the-shelf large language models (LLMs) for cybersecurity applications. In particular, automated vulnerability assessment and penetration-testing

Authors’ Contact Information: [Andreas Happe](#), andreas.happe@tuwien.ac.at, TU Wien, Vienna, Austria; [Jürgen Cito](#), juergen.cito@tuwien.ac.at, TU Wien, Vienna, Austria.

© 2025 Copyright held by the owner/author(s).

Manuscript submitted to ACM

Manuscript submitted to ACM

1

have emerged as promising fields of investigation to remediate challenges associated with limited human expertise and high operational costs in traditional red-teaming and penetration-testing exercises [15]. Penetration testing is critical for organizations to validate defenses and uncover latent vulnerabilities. *Assumed Breach* assessments simulate an attacker that has already breached the perimeter and is within the target company’s internal network. They are particularly relevant given that real-life cyberattacks, such as ransomware incidents, often mirror these internal threat scenarios.

In such contexts, autonomous systems that emulate adversarial behavior become invaluable not only for proactive risk assessment but also for preparing defensive blue teams to counter increasingly sophisticated automated attackers. As noted in earlier work [58], while synthetic benchmarks have provided useful insights, the complexity and dynamic nature of real-world networks necessitate evaluations within realistic environments. Our study focuses on Microsoft Active Directory networks—ubiquitous in enterprise settings and frequent targets of ransomware attacks—where the need for more efficient testing is acute.

Existing proof-of-concept prototypes, such as PentestGPT [7] and HackingBuddyGPT [14, 18], have paved the way toward automated penetration testing. However, these systems are often constrained either by partial automation or by a narrow focus on targeting single host scenarios, whereas this work investigates more complex multi-host networks.

In this paper, we investigate a critical question: Is an automated LLM-driven assumed breach simulation a feasible and effective approach for compromising enterprise networks? Building on best practices observed in earlier studies, we present a novel prototype, shown in Figure 1, that allows LLMs to autonomously perform most phases of the penetration testing lifecycle, spanning reconnaissance, credential access, and discovery phases—as delineated by the MITRE ATT&CK¹ [49] framework—with initial explorations into lateral movement and execution. We empirically evaluate the offensive security capabilities of five different LLMs, including open-weight, reasoning, and locally-run models. Our work constitutes the first demonstration of a fully autonomous, LLM-driven framework capable of compromising accounts within a real-life testbed, namely the Game of Active Directory (GOAD)². The analysis of the evaluated LLMs’ strengths and weaknesses lays the groundwork for future software engineering research into using LLMs for security tasks.

1.1 Illustrative Example

We want to use a realistic example to set the stage:

You are an IT employee of a small enterprise (SME) that handles sensitive customer data. Given your concern about data security, you proposed to verify your company’s security posture with a network security assessment performed by professional external penetration-testers. The test was estimated to take seven days [15], you negotiated a favorable hourly rate of \$180 (Section 2.2.2) yielding a total projected cost of \$10,080. Unfortunately, the company’s management was not able to allocate the required resources, so the security assignment was postponed to next year.

Fast-forward a couple of weeks. On a Monday, you enter the enterprise offices to find your company to have become the target of a ransomware group. All data has been encrypted and a hefty ransom is demanded. The perpetrator was able to traverse through your company network, gain access to multiple user accounts, and finally was able to encrypt all your data including your backups. In addition, the

¹<https://attack.mitre.org/matrices/enterprise/>

²<https://github.com/Orange-Cyberdefense/GOAD>

```

Tool Call(s)
SshExecuteTool: impacket-GetNPUsers ESSOS.LOCAL/missandei -dc-ip 192.168.56.12 -no-pass

Result for impacket-GetNPUsers ESSOS.LOCAL/missandei -dc-ip 192.168.56.12 -no-pass
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for missandei
/usr/share/doc/python3-impacket/examples/GetNPUsers.py:165: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a
future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
$krb5asrep$23$missandei@ESSOS.LOCAL:09d699fa6a8df65d343701325b2329c9$fl2e5c8b1e848e521d765febbda37f0fd861e4272680d393b6bbfcc15db20ef8a243c0f242526c247d5
a8e91454574a32e1d6a39a99feaf99c2eb1b8a55e9915b28ef30981d52fa8921f09e175bbec019ca95bc16de2339fc90792f8a8892d758ac15a48a05facc779056835a1a520e47b13b14c66f
bf0944d497f38e925812ed77b8d99f971da3515ec4cd0e5e743595b1a7b58094da487d680bbf10f84ced485f7789a2cdb80c7b285937ecd747e8df6529d01bed8ee599ac845d13e9a6d997c1
51e5632edb9faeea614afe50adb77dd2a3894a1464e88207e7e8d08eb91495281f3c4811d32518

Tool Call(s)
SshExecuteTool: echo
'$krb5asrep$23$missandei@ESSOS.LOCAL:09d699fa6a8df65d343701325b2329c9$fl2e5c8b1e848e521d765febbda37f0fd861e4272680d393b6bbfcc15db20ef8a243c0f242526c247d
5a8e91454574a32e1d6a39a99feaf99c2eb1b8a55e9915b28ef30981d52fa8921f09e175bbec019ca95bc16de2339fc90792f8a8892d758ac15a48a05facc779056835a1a520e47b13b14c66f
bf0944d497f38e925812ed77b8d99f971da3515ec4cd0e5e743595b1a7b58094da487d680bbf10f84ced485f7789a2cdb80c7b285937ecd747e8df6529d01bed8ee599ac845d13e9a6d997c
151e5632edb9faeea614afe50adb77dd2a3894a1464e88207e7e8d08eb91495281f3c4811d32518' > /root/missandei_hash.txt

Result for echo '$krb5asrep$23$missandei@ESSOS.LOCAL:09d699fa6a8df65d343701325b2329c9$fl2e5c8b1e848e521d765febbda37f0fd861e4272680d393b6bbfcc15db20ef8a
243c0f242526c247d5a8e91454574a32e1d6a39a99feaf99c2eb1b8a55e9915b28ef30981d52fa8921f09e175bbec019ca95bc16de2339fc90792f8a8892d758ac15a48a05facc779056835a1a520e47b13b14c66f
bf0944d497f38e925812ed77b8d99f971da3515ec4cd0e5e743595b1a7b58094da487d680bbf10f84ced485f7789a2cdb80c7b285937ecd747e8df6529d01bed8ee599ac845d13e9a6d997c
151e5632edb9faeea614afe50adb77dd2a3894a1464e88207e7e8d08eb91495281f3c4811d32518' > /root/missandei_hash.txt

Tool Call(s)
SshExecuteTool: john --wordlist=/usr/share/wordlists/rockyou.txt /root/missandei_hash.txt

Result for john --wordlist=/usr/share/wordlists/rockyou.txt /root/missandei_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP etype 17/18/23 [MD4 HMAC-MD5 RC4 / PBKDF2 HMAC-SHA1 AES 256/256 AVX2 8x])
Will run 2 OpenMP threads
Press Ctrl-C to abort, or send SIGUSR1 to john process for status
fr3edom ($krb5asrep$23$missandei@ESSOS.LOCAL)
lg 0:00:00:01 DONE (2025-01-29 02:59) 0.6024g/s 1081Kp/s 1081Kc/s 1081Kc/s franciene..foyllover
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

Executor Result
The password for the user 'missandei@ESSOS.LOCAL' has been successfully cracked using John the Ripper. The password is 'fr3edom'.

Key Findings:
- The Kerberos hash for 'missandei@ESSOS.LOCAL' was successfully extracted and cracked.
- The password for 'missandei@ESSOS.LOCAL' is 'fr3edom'.
- This provides us with a valid set of credentials for further penetration testing activities within the ESSOS.LOCAL domain.

```

Fig. 1. Our prototype combines two Active Directory attacks (AS-REP Kerberos Roasting, following up by password-cracking) to compromise a user account without human interaction (Experiment run can be found in [run-20250129-085237.json](#)).

ransomware group threatens to release all the sensitive customer data, making your company potentially liable for additional damages.

The damage to the company is manifold: the monetary damage of the paid ransom, the disruption of operation during the incident, the loss of customer trust. Many companies, especially SMEs and NGOs, are not able to recover from such a ransomware incident. Being able to reduce the price of security-testing would have prevented this incident.

This short example is a typical ransomware incident as analyzed by academia [25, 26] and industry [1, 35]. Current industry reports estimate the direct damages inflicted by ransomware in 2025 at \$6.5m per hour, with incident rates expected to rate to an incident every two seconds by 2031 [35].

New Zealand’s Computer Emergency Response Team (CERT) [39] separates ransomware attacks into three phases:

- During *Initial Access* the attacker gains access to the company network. Typically this is achieved using leaked VPN credentials or through social engineering. Both industry [1] and academia [19] show the uptake of LLMs for performing these tasks.
- During the *Consolidation and Preparation* phase, the attacker moves through the breached internal network. They try to gain access to as many accounts and systems as possible. Penetration-tests, or more specifically *Assumed*

Manuscript submitted to ACM

Breach Simulations (Section 2.2), emulate this activity to find vulnerabilities to allow defenders to mitigate them before “real” attackers exploit them. This is the area on which our research focuses.

- Finally, during the *Impact on Target* phase, attackers perform their goal, typically performing industry espionage, encrypting, or destroying data.

1.2 Motivation

Attackers will gain access to internal organization networks. Modern defensive techniques, e.g. Zero-Trust Architectures [59], accept this and try to minimize the potential impact that an attacker can inflict within internal networks. Typically, organizations perform *Assumed Breach Simulations* (Section 2.2) to find potential security vulnerabilities, and subsequently fix them. The *Simulation* in *Assumed Breach Simulation* stands for simulating attackers; all performed operations are real hacking operations performed against the live organization network. This does not happen regularly due to the high cost of performing security-testing.

The motivation for our research is multi-fold:

- to evaluate the capabilities of LLMs to perform Assumed Breach Simulations against live networks. This implies that we will use a realistic and complex testbed for our *Capability Evaluation*.
- to investigate the costs of using LLM-powered security testing. Are they a viable alternative for SME and NGOs which often cannot afford human penetration-testers?
- to raise awareness about LLM’s offensive capabilities, esp. with LLM providers and LLM creators. If off-the-shelf LLMs are capable of penetration-testing, future LLMs should include safe rails to prevent abuse.

1.3 Contributions

This paper includes the following contributions:

- **A Novel Autonomous Prototype for Penetration-Testing.** We introduce a novel prototype that autonomously conducts complex penetration-tests on live enterprise networks using the ubiquitous Microsoft Active Directory. Our system is designed to automate a complex and human-intensive software security task.
- **Comprehensive Evaluation of LLM Capabilities in Real-Life Scenarios.** We provide a comprehensive evaluation of LLM capabilities in penetration-testing, detailing both strengths and limitations in real-life contexts. The deliberate choice of a “messy” live testing environment addresses known concerns about the limitations of synthetic testbeds for real-life security impact evaluations [34, 58].
- **Systematic Quantitative and Qualitative Analysis with Expert Insights.** We systematically analyze quantitative metrics and integrate qualitative insights gathered from security experts. Our multi-faceted approach, combining automated data with human expert analysis, enhances the depth and validity of our findings. The validation of the prototype’s activities against established cybersecurity frameworks like MITRE ATT&CK links observed behaviors to recognized industry standards and grounds our research in practical, real-world software security engineering.
- **Investigating the Impact of Reasoning LLMs.** To the best of our knowledge this is the first paper that applies cutting-edge Reasoning LLMs to the problem of performing automated penetration-testing.

While we have chosen a scenario from the security domain for our evaluation, the used LLM architecture and techniques are domain-agnostic and can be used for improving the autonomous usage of LLMs in non-related domains.

1.4 Ethics Statement

Given that security tools inherently possess dual-use characteristics, we need to address ethical considerations. In line with community consensus in the security domain [17], we advocate for transparent, open-source dissemination of our work. Open security tooling ultimately enhances collective cybersecurity. To facilitate future discussion we release our prototype, all captured raw log data, and our intermediate analysis of the logs as open-source on GitHub.

1.5 Source Code and Analysis Package

All source code artifacts, captured logs, screenshots, etc. are publicly available through GitHub at <https://github.com/andreashappe/cochise>. The prototype version used for the experiment runs detailed within this paper was 3084bccdd99f85e5ce324f25d0d49f80439fd5382, commit b3b00e6340f58f0af630759522af47903f07cd80 contains all used log data and analysis scripts used within this paper.

2 Background & Related Work

Our background sections opens up with information about enterprise networks and commonly performed penetration-testing approaches, subsequently investigates improvements in LLM-guided task planning, contemporary application of these improvements upon autonomous penetration-testing, and closes with differentiating our work with mentioned approaches.

2.1 Enterprise Networks and Common Attacks

Microsoft Active Directory (AD) was introduced in 1999 and made public, together with *Microsoft Windows Server 2000*, on 17.2.2000. It has since become the predominant means of managing user information in enterprise computer networks. Industry estimates indicate that over 90% of Global Fortune 1000 companies are using AD as their primary means for user authentication and authorization [32].

2.1.1 Active Directory Structure. A *Domain* represents a “database with records about network service-things such as computers, users, groups, and other things that use, support, or exist on a network” [28]. This database is used for authentication and authorization within the respective enterprise network. It is stored and synchronized between one or more *Domain Controllers* (DC).

Multiple domains can be linked in a hierarchical *Domain Tree*. This is commonly done to simplify administration and to model relationships between departments within a single organization. At the highest level, the *Active Directory Forest* is a collection of trees with a standard global catalog, directory schema, and logical structure. A forest also establishes a trust boundary and thus penetration-tests are often scoped on the forest level.

On a lower level, an AD uses multiple network protocols. For exchange of authorization information, the NTLM and Kerberos protocols are used. LDAP can be used to query the AD for user information directly. Typical services deployed within an AD are the *Microsoft SQL Server* (MSSQL), *Microsoft Exchange*, or the *Microsoft Internet Information Server* (IIS, a web- and application server).

2.1.2 Common Active Directory Attacks. Given its ubiquity in enterprise networks, AD has become the prime attack target [3, 60] with industry reports estimating that “Fifty percent of organizations have experienced an Active Directory attack in the last two years, with 40% of those attacks successful because the adversary was able to exploit poor Active